

# Ejercicio 1: Algo huele mal

Indique qué malos olores se presentan en los siguientes ejemplos.

## 1.1 Protocolo de Cliente

La clase `Cliente` tiene el siguiente protocolo. ¿Cómo puede mejorarlo?

```
/**
 * Retorna el límite de crédito del cliente
 */
public double lmtCrdt() {...}

/**
 * Retorna el monto facturado al cliente desde la fecha f1 a la fecha f2
 */
protected double mtFcE(LocalDate f1, LocalDate f2) {...}

/**
 * Retorna el monto cobrado al cliente desde la fecha f1 a la fecha f2
 */
private double mtCbE(LocalDate f1, LocalDate f2) {...}
```

Rta.:

```
public double getLimiteCrediticio() {...}

public double getMontoFacturado(LocalDate fechaInicio, LocalDate fechaFin) {...}

public double getMontoCobrado(LocalDate fechaInicio, LocalDate fechaFin) {...}
```

## 1.2 Participación en proyectos

Al revisar el siguiente diseño inicial (Figura 1), se decidió realizar un cambio para evitar lo que se consideraba un mal olor. El diseño modificado se muestra en la Figura 2. Indique qué tipo de cambio se realizó y si lo considera apropiado. Justifique su respuesta.

**Diseño inicial:**

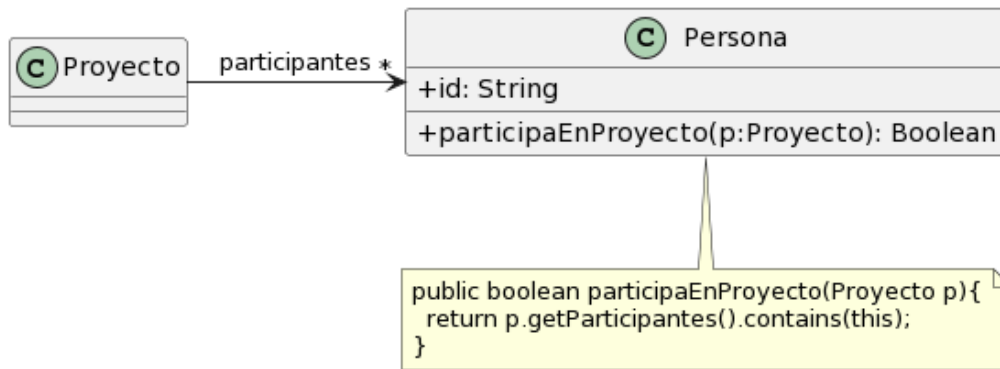


Figura 1: Diagrama de clases del diseño inicial.

**Diseño revisado:**

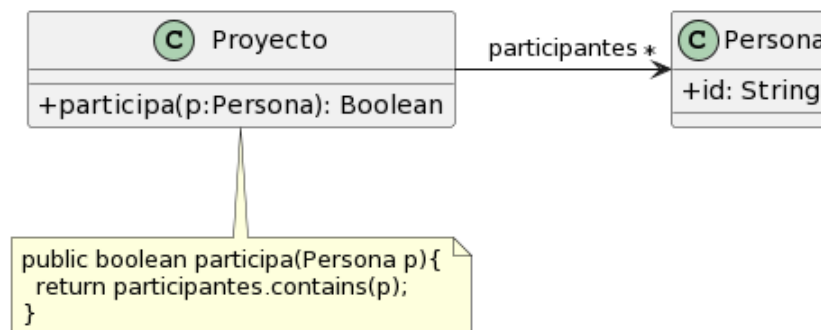


Figura 2: Diagrama de clases modificado.

**Rta.:**

**Cambios realizados:**

- Move Method “participaEnProyecto”: desde Persona hacia Proyecto
  - El problema es que Persona tenía envidia de atributos sobre Proyecto.
  - Proyecto tenía que exponer “participantes”.

**Falta:**

- Poner la v.i. “id” de Persona como privada.
- Poner la v.i. “participantes” de Proyecto como privada.

## 1.3 Cálculos

Analice el código que se muestra a continuación. Indique qué *code smells* encuentra y cómo pueden corregirse.

```

public void imprimirValores() {
    int totalEdades = 0;
    double promedioEdades = 0;
    double totalSalarios = 0;

    for (Empleado empleado : personal) {
        totalEdades = totalEdades + empleado.getEdad();
    }
}

```

```

        totalSalarios = totalSalarios + empleado.getSalario();
    }
    promedioEdades = totalEdades / personal.size();

    String message = String.format("El promedio de las edades es %s y el total de salarios es %s",
    promedioEdades, totalSalarios);

    System.out.println(message);
}

```

**Rta.:**

**Code smells y sus soluciones:**

- Bucles imperativos: reemplazar el bucle for por el uso de Streams sobre *personal*.
- Uso de variables temporales: aplicar el refactoring Replace Temp with Query, informando métodos en vez de temporales.
- Long Method: aplicar Extract Method y descomponer el método *imprimirValores* en 3 métodos para imprimir el total de edades, el promedio de edades y el total de salarios.

## Ejercicio 2: Iteradores circulares

Dada la siguiente implementación:

**CharRing.java**

```

1  public class CharRing extends Object {
2      char[] source;
3      int idx;
4
5      public CharRing(String srcString) {
6          char result;
7          source = new char[srcString.length()];
8          srcString.getChars(0, srcString.length(), source, 0);
9          result = 0;
10         idx = result;
11     }
12
13     public char next() {
14         int result;
15         if (idx >= source.length)
16             idx = 0;
17         result = idx++;
18         return source[result];
19     }
20 }

```

### Tareas:

- 1) Se quiere aplicar el refactoring *Rename Variable* sobre la variable **result** que se usa en la línea 18 con el nuevo nombre **currentPosition**.
- 2) ¿Cómo queda el código final y qué inconveniente se podría encontrar?

Rta.:

```
1 public class CharRing extends Object {
2     char[] source;
3     int idx;
4
5     public CharRing(String srcString) {
6         char result;
7         source = new char[srcString.length()];
8         srcString.getChars(0, srcString.length(), source, 0);
9         result = 0;
10        idx = result;
11    }
12
13    public char next() {
14        int currentPosition;
15        if (idx >= source.length)
16            idx = 0;
17        currentPosition = idx++;
18        return source[currentPosition];
19    }
```

Al renombrar la variable de la línea 18, se debe renombrar también en la línea 14 y 17.

## Ejercicio 3: Iteradores circulares bis

Se cuenta con las siguientes implementaciones de iteradores circulares, las cuales presentan implementaciones similares.

CharRing.java

```

1  public class CharRing {
2      private char[] source;
3      private int idx;
4
5      public CharRing(String src) {
6          source = src.toCharArray();
7          idx = 0;
8      }
9
10     public char next() {
11         if (idx >= source.length)
12             idx = 0;
13         return source[idx++];
14     }
15 }

```

|              |
|--------------|
| IntRing.java |
|--------------|

```

1  public class IntRing {
2      private int[] source;
3      private int idx;
4
5      public IntRing(int[] src) {
6          source = src;
7          idx = 0;
8      }
9
10     public int next() {
11         if (idx >= source.length)
12             idx = 0;
13         return source[idx++];
14     }
15 }

```

## Tareas:

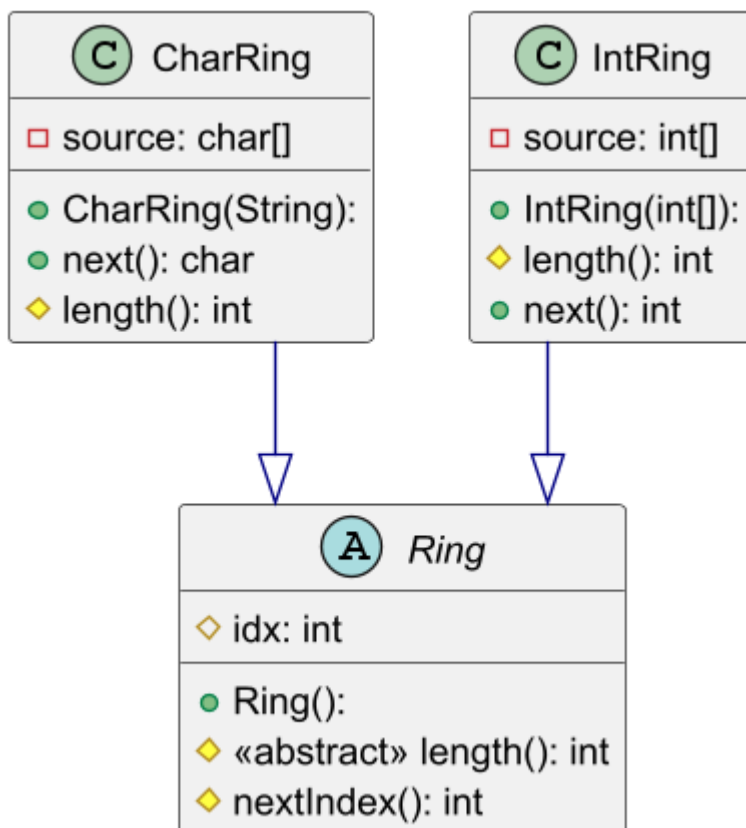
- 1) Diseñe e implemente Test de Unidad para las clases CharRing e IntRing. Asegúrese de que los test pasen.
- 2) Aplique el refactoring **Extract Superclass**. Detalle cada uno de los pasos intermedios que son necesarios para poder aplicar correctamente este refactoring.
- 3) Verifique que los tests definidos en el paso 1 sigan funcionando correctamente.
- 4) Realice un diagrama de clases UML con el diseño refactorizado.

Rta.:

Para aplicar el **Extract Superclass** del punto 2...

1. Creo la superclase abstracta.
2. Subo las variables de instancia en común entre CharRing e IntRing (tdx) y la establezco como protegida (pull up field).
3. En la superclase creo un método abstracto para que las subclases calculen la longitud de sus iteradores circulares (template method).
4. En las subclases creo un método concreto que calcula el siguiente índice (extract method).
  - a. Al ser iguales, extraigo el método de las subclases y lo subo a la superclase (pull up method).

UML final:



## Ejercicio 4: Alcance en Redes Sociales

Una nueva red social está desarrollando un sistema para modelar perfiles y publicaciones, y medir su alcance como parte del algoritmo de relevancia. Cada publicación acumula reacciones de los usuarios, y cada perfil consolida el alcance de sus publicaciones amplificándolo según si está verificado o no. El código es el siguiente:

**Publicacion.java**

```

1  package redesocial;
2  public class Publicacion {
3      private String texto;
4      private int likes;
5      public Publicacion(String texto) {
6          this.texto = texto;
7          this.likes = 0;
8      }
9      public void darLike() { likes++; }
10     public void darDislike() { likes--; }
11     private int procesar() {
12         return likes * 3;
13     }
14     public int calcular() {
15         return procesar() * 10;
16     }
17 }

```

#### Perfil.java

```

1  package redesocial;
2  import java.util.ArrayList;
3  public class Perfil {
4      private boolean verificado;
5      private ArrayList<Publicacion> publicaciones;
6      public Perfil(boolean verificado) {
7          this.verificado = verificado;
8          this.publicaciones = new ArrayList<>();
9      }
10     public void agregarPublicacion(Publicacion p) { publicaciones.add(p); }
11     private int bonus() { return verificado ? 2 : 1; }
12     private int alcanceDePublicaciones() {
13         return publicaciones.stream().mapToInt(p -> p.calcular()).sum();
14     }
15     public int calcular() {
16         return alcanceDePublicaciones() * bonus();
17     }
18 }

```

#### Tarea:

Liste cada uno de los cambios necesarios, indicando archivo y línea afectados, para cada uno de los siguientes refactorings:

1. Rename method: **procesar** (referenciado en línea 11 de Publicacion.java) por **impacto**

2. Rename method: **calcular** (referenciado en línea 14 de Publicacion.java) por **alcance**
3. Rename method: **calcular** (referenciado en línea 15 de Perfil.java) por **alcance**
4. Rename parameter: el parámetro “**p**” del método **agregarPublicacion** (línea 10 de Perfil.java) por “**publicacion**”

**Rta.:**

1. [Publicacion.java; línea 11, columna 16] procesar → impacto  
[Publicacion.java; línea 15, columna 15] procesar → impacto
2. [Publicacion.java; línea 14, columna 15] calcular → alcance
3. [Perfil.java; línea 15, columna 15] calcular → alcance  
[Perfil.java; línea 13, columna 53] calcular → alcance
4. [Perfil.java; línea 10, columna 47] p → publicacion  
[Perfil.java; línea 10, columna 70] p → publicacion

## Ejercicio 5: Productos

Se cuenta con un sistema que maneja una jerarquía de productos que pueden ser estadías de hoteles o alquileres de autos. En el siguiente [material adicional](#) puede descargar un proyecto con las clases implementadas. Observe que el proyecto cuenta con test unitarios que verifican el funcionamiento correcto de la aplicación. A continuación se muestra un extracto del código:

**HotelStay.java**



```
1 public class HotelStay extends Product {
2     public double cost;
3     private TimePeriod timePeriod;
4     private Hotel hotel;
5
6     public HotelStay(double cost, TimePeriod timePeriod, Hotel hotel) {
7         this.cost = cost;
8         this.timePeriod = timePeriod;
9         this.hotel = hotel;
10    }
11
12    public LocalDate startDate() {
13        return this.timePeriod.start();
14    }
15
16    public LocalDate endDate() {
17        return this.timePeriod.end();
18    }
19
20    public double priceFactor() {
21        return this.cost / this.price();
22    }
23
24    public double price() {
25        return this.timePeriod.duration() * this.hotel.nightPrice() * this.hotel.discountRate();
26    }
27 }
28
```

CarRental.java

```

1  public class CarRental extends Product {
2      public double cost;
3      private TimePeriod timePeriod;
4      private Company company;
5
6      public CarRental(double cost, TimePeriod timePeriod, Company company) {
7          this.cost = cost;
8          this.timePeriod = timePeriod;
9          this.company = company;
10     }
11
12     public LocalDate startDate() {
13         return this.timePeriod.start();
14     }
15
16     public LocalDate endDate() {
17         return this.timePeriod.end();
18     }
19
20     public double price() {
21         return this.company.price() * this.company.promotionRate();
22     }
23
24     public double cost() {
25         return this.cost;
26     }
27 }

```

## Tareas:

- La variable “cost” está declarada como pública, lo que rompe el encapsulamiento de la clase. Utilice el refactoring **Encapsulate Field** y describa brevemente los pasos llevados a cabo. Verifique que los tests provistos sigan funcionando. Discuta con un ayudante:
  - ¿Es correcto modificar alguno de los tests para que el código refactorizado funcione?
  - En caso de que el test falle, ¿qué situación está representando este test?
- Utilice el refactoring **Rename Field** en el método priceFactor(), para que la variable “cost” se pase a llamar “quote”. Verifique que los tests provistos sigan funcionando. Discuta con un ayudante: ¿en este caso, es necesario modificar alguno de los tests para que el código refactorizado funcione?
- Se quiere aplicar el refactoring **Pull Up Method** para subir los métodos startDate() y endDate() a la superclase Product. ¿Es posible hacerlo en el código anterior? Justifique su respuesta basándose en las precondiciones del refactoring vistas en la teoría y en el libro de Refactoring de Martin Fowler.
- Mencione y aplique los refactorings necesarios para poder hacer Pull Up Method.

5. Aplique Pull Up Method para subir los métodos a la superclase Product. Verifique que los tests provistos sigan funcionando.
6. Observe los métodos price() en CarRental.java (línea 21) y en HotelStay.java (línea 25). Identifique los Code Smell presentes. Luego aplique los refactoring correspondientes para eliminarlos. Verifique que los tests provistos sigan funcionando.

**Rta.:**

1. Si marco *costo* como protegido no hay problema. Sin embargo, la bibliografía y las teorías enuncian que, al aplicar Encapsulate Field, en la mayoría de los casos se deben marcar como privados los campos que están en juego. Si se tiene en cuenta el último caso, los pasos a llevar a cabo son: 1. establecer el alcance de *cost* en CarRental y HotelStay como privado; 2. crear getter para *cost*
  - a. En este caso yo creo que sí es correcto
  - b. ?
2. En un principio, al cambiar el nombre de todos los campos dentro de HotelStay que se llamen “cost” a “quote”, no hace falta modificar los tests. Sin embargo, hay que aplicar Rename Method a getCost() para comunicar claramente la intención del método (cambiar a getQuote()). Gracias a este último caso, hay que modificar el test.
3. Según Fowler: “Para subir un método a una superclase, los cuerpos y firmas de los métodos en las subclases deben ser idénticos. Además, no debe haber conflictos con otros métodos en la superclase y todos los elementos a los que accede el método deben ser accesibles desde dicha superclase”. Si seguimos esto al pie de la letra, se viola el enunciado “todos los elementos a los que accede el método deben ser accesibles desde dicha superclase” ya que el campo *timePeriod* es privado. Habría que hacer un Pull Up Field de *timePeriod*. Además, habría modificar los constructores de las subclases para que *Product* asigne *timePeriod* con super(). Por otro lado, hay que definir el alcance de *timePeriod* como protegido para que price() en HotelStay no entre en conflicto.
4. Viendo los métodos de las subclases, ninguno cumple con el requerimiento de que los cuerpos y firmas sean idénticos. Por lo tanto, no hace falta hacer ningún otro Pull Up Method.
5. Los tests siguen funcionando.
6. price() de CarRental:
  - a. Envidia de atributos: CarRental está más interesado en los atributos de Company que en los suyos.
  - b. Solución: Move Method: hacer un método en Company que se encargue de calcular su precio final.price() de HotelStay:
  - a. Envidia de atributos: HotelStay está más interesado en los atributos de Hotel que en los suyos.

b. Solución: Move Method: hacer un método en Hotel que se encargue de calcular su precio final en base a la duración del periodo de tiempo recibido por parámetro.  
Una vez aplicados los cambios, los tests siguen funcionando.

## Ejercicio 6:

Para cada una de las siguientes situaciones, realice en forma iterativa los siguientes pasos:

- (i) indique el mal olor,
  - (ii) indique el refactoring que lo corrige,
  - (iii) aplique el refactoring, mostrando el resultado final (código y/o diseño según corresponda).
- Si vuelve a encontrar un mal olor, retorne al paso (i).

### 6.1 Empleados

```
public class EmpleadoTemporario {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    public double horasTrabajadas = 0;
    public int cantidadHijos = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico
            + (this.horasTrabajadas * 500)
            + (this.cantidadHijos * 1000)
            - (this.sueldoBasico * 0.13);
    }
}
```

```
public class EmpleadoPlanta {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    public int cantidadHijos = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico
            + (this.cantidadHijos * 2000)
            - (this.sueldoBasico * 0.13);
    }
}
```

```

public class EmpleadoPasante {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico - (this.sueldoBasico * 0.13);
    }
}

```

**Rta.:**

- i.
  - A. Public fields
  - B. Duplicate code: nombre, apellido, sueldoBasico, (this.sueldoBasico \* 0.13).
  - C. Magic numbers: 0.13, 500, 1000, 2000
  - D. Comments
- ii.
  - A. Encapsulate fields
  - B. Pull up method, Pull up fields, Form template method
  - C. Replace Magic Number with Symbolic Constant

## 6.2 Juego

```

public class Juego {
    // .....
    public void incrementar(Jugador j) {
        j.puntuacion = j.puntuacion + 100;
    }
    public void decrementar(Jugador j) {
        j.puntuacion = j.puntuacion - 50;
    }
}

public class Jugador {
    public String nombre;
    public String apellido;
    public int puntuacion = 0;
}

```

**Rta.:**

- i.
  - A. Public fields
  - B. Feature envy
  - C. Magic numbers
- ii.
  - A. Encapsulate fields
  - B. Move method: incrementar y decrementar en clase Jugador
  - C. Replace magic number with symbolic constants

## 6.3 Publicaciones



```
/**
 * Retorna los últimos N posts que no pertenecen al usuario user
 */
public List<Post> ultimosPosts(Usuario user, int cantidad) {

    List<Post> postsOtrosUsuarios = new ArrayList<Post>();
    for (Post post : this.posts) {
        if (!post.getUsuario().equals(user)) {
            postsOtrosUsuarios.add(post);
        }
    }

    // ordena los posts por fecha
    for (int i = 0; i < postsOtrosUsuarios.size(); i++) {
        int masNuevo = i;
        for (int j = i + 1; j < postsOtrosUsuarios.size(); j++) {
            if (postsOtrosUsuarios.get(j).getFecha().isAfter(
                postsOtrosUsuarios.get(masNuevo).getFecha())) {
                masNuevo = j;
            }
        }
        Post unPost = postsOtrosUsuarios.set(i, postsOtrosUsuarios.get(masNuevo));
        postsOtrosUsuarios.set(masNuevo, unPost);
    }

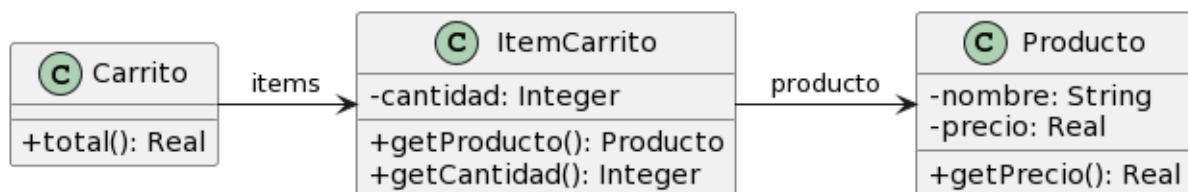
    List<Post> ultimosPosts = new ArrayList<Post>();
    int index = 0;
    Iterator<Post> postIterator = postsOtrosUsuarios.iterator();
    while (postIterator.hasNext() && index < cantidad) {
        ultimosPosts.add(postIterator.next());
    }
    return ultimosPosts;
}
```

Rta.:

i.

- A. Imperative loops
- B. Long method
- C. Comments
- ii. Replace loop with pipeline

## 6.4 Carrito de compras



```
public class Producto {
    private String nombre;
    private double precio;

    public double getPrecio() {
        return this.precio;
    }
}
```

```
public class ItemCarrito {
    private Producto producto;
    private int cantidad;

    public Producto getProducto() {
        return this.producto;
    }

    public int getCantidad() {
        return this.cantidad;
    }
}
```

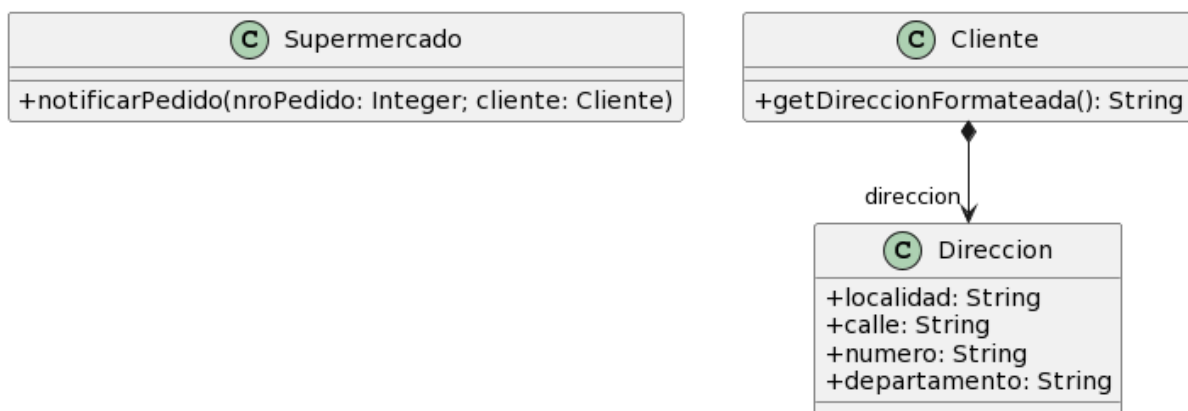
```
public class Carrito {
    private List<ItemCarrito> items;

    public double total() {
        return this.items.stream()
            .mapToDouble(item ->
                item.getProducto().getPrecio() * item.getCantidad())
            .sum();
    }
}
```

Rta.:

- i.
  - A. Public field en Carrito
  - B. Feature envy en total() de Carrito
  - C. Message chain
- ii.
  - A. Encapsulate field
  - B. Move method
  - C. Extract method y Move method

## 6.5 Envío de pedidos



```
public class Supermercado {
    public void notificarPedido(long nroPedido, Cliente cliente) {
        String notificacion = MessageFormat.format("Estimado cliente, se le informa que hemos recibido
su pedido con número {0}, el cual será enviado a la dirección {1}", new Object[] { nroPedido,
cliente.getDireccionFormateada() });

        // lo imprimimos en pantalla, podría ser un mail, SMS, etc..
        System.out.println(notificacion);
    }
}
```

```
public class Cliente {
    public String getDireccionFormateada() {
        return
            this.direccion.getLocalidad() + ", " +
            this.direccion.getCalle() + ", " +
            this.direccion.getNumero() + ", " +
            this.direccion.getDepartamento()
    }
}
```

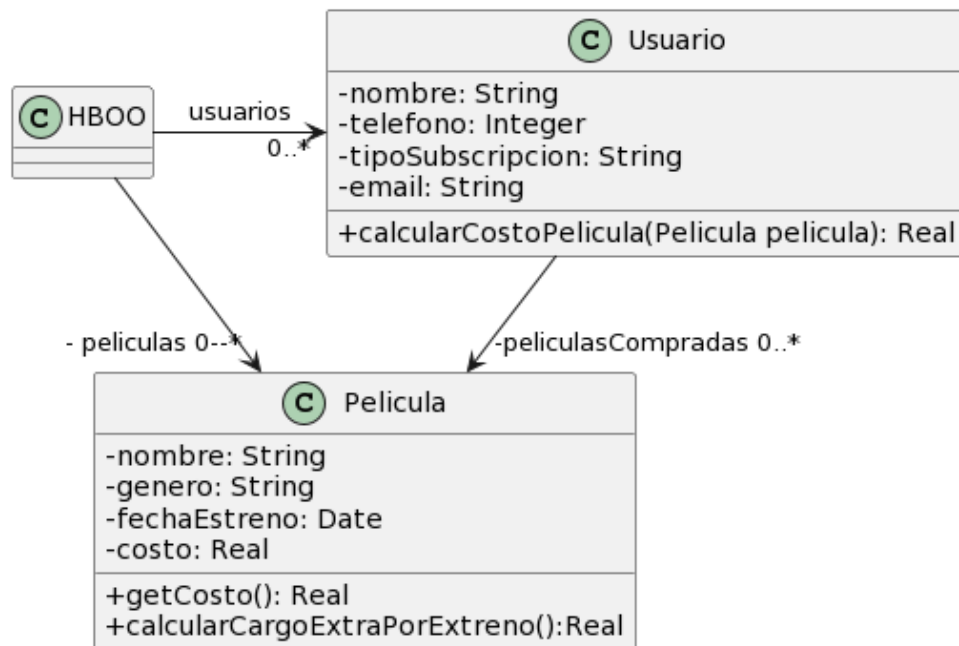
Rta.:

- i.
  - A. Feature envy en getDireccionFormateada() de Cliente



- ii.
  - B. Public fields en Direccion
  - C. Comments en Supermercado
- A. Move method
  - B. Encapsulate fields
  - C. Extract method y Rename method

## 6.6 Películas



```

public class Usuario {
    String tipoSubscripcion;
    // ...

    public void setTipoSubscripcion(String unTipo) {
        this.tipoSubscripcion = unTipo;
    }

    public double calcularCostoPelicula(Pelicula pelicula) {
        double costo = 0;
        if (tipoSubscripcion=="Basico") {
            costo = pelicula.getCosto() + pelicula.calcularCargoExtraPorEstreno();
        }
        else if (tipoSubscripcion== "Familia") {
            costo = (pelicula.getCosto() + pelicula.calcularCargoExtraPorEstreno()) * 0.90;
        }
        else if (tipoSubscripcion=="Plus") {
            costo = pelicula.getCosto();
        }
        else if (tipoSubscripcion=="Premium") {
            costo = pelicula.getCosto() * 0.75;
        }
    }
}
  
```

```

    }
    return costo;
}
}

```

```

public class Pelicula {
    LocalDate fechaEstreno;
    // ...

    public double getCosto() {
        return this.costo;
    }

    public double calcularCargoExtraPorEstreno(){
        // Si la Película se estrenó 30 días antes de la fecha actual, retorna un cargo de 0$, caso
        // contrario, retorna un cargo extra de 300$
        return (ChronoUnit.DAYS.between(this.fechaEstreno, LocalDate.now()) > 30 ? 0 : 300;
    }
}

```

**Rta.:**

- i.
  - A. Lazy class en HBOO
  - B. Switch statements en calcularCostoPelicula() de Usuario
  - C. Comments en Pelicula
- ii.
  - A. Eliminar a HBOO, no tiene razón de ser
  - B. Replace Conditional with Polymorphism
  - C. Extract method (un metodo que calcule los dias entre el estreno y ahora) y Rename method